

Classes Objetos e Cópias Independentes

Caderno de aprendizagem e exercícios

Java com BlueJ | Documento do aluno

Objetivo do trabalho

Neste caderno vais construir uma pequena Academia de Criaturas Mágicas. O objetivo é compreender como se criam objetos, como um objeto pode receber outro objeto como atributo e como se produz uma cópia verdadeiramente independente através de um método clone.

No final deves conseguir

- Distinguir uma classe de um objeto e identificar o estado de cada instância.
- Criar atributos, construtores, métodos get e set e o método toString.
- Usar um objeto de outra classe como atributo.
- Distinguir a cópia de uma referência da criação de um novo objeto.
- Construir métodos clone que efetuem cópias independentes.
- Criar uma cópia profunda de um objeto que contém outros objetos e uma ArrayList.

Ficheiros do projeto BlueJ

Cria um projeto chamado AcademiaMagica. Cada classe pública deve ficar num ficheiro com o mesmo nome da classe.

- CriaturaMagica.java
- Treinador.java
- GuildaMagica.java
- MissaoMagica.java
- AppAcademia.java

Parte 1 Classes objetos e estado

Conceitos essenciais

Uma classe descreve os dados e os comportamentos de um tipo de objeto. Um objeto é uma instância concreta dessa classe. Duas criaturas criadas com new podem ter os mesmos valores, mas continuam a ser objetos diferentes.

```
CriaturaMagica a = new CriaturaMagica("Faísca", "Dragão", 4, 80);  
CriaturaMagica b = new CriaturaMagica("Faísca", "Dragão", 4, 80);
```

Os objetos a e b têm o mesmo estado inicial, mas ocupam referências diferentes. A expressão a == b compara as referências, não os valores dos atributos.

Exercício 1 A criatura inicial

1. Cria a classe CriaturaMagica com os atributos privados nome, especie, nivel e energia.
2. Cria um construtor completo que receba os quatro valores.
3. Cria um construtor sem parâmetros com os valores Sem nome, Desconhecida, 1 e 100.
4. Cria os métodos get e set para todos os atributos.
5. Valida o nível entre 1 e 10 e a energia entre 0 e 100.
6. Cria um método toString que apresente todos os dados.

Exercício 2 Comportamentos

7. Cria o método treinar. Deve aumentar um nível, sem ultrapassar 10, e gastar 10 pontos de energia.
8. Cria o método descansar. Deve recuperar 20 pontos de energia, sem ultrapassar 100.
9. Cria o método estaPronta. Deve devolver true quando a energia for pelo menos 30.

Exercício 3 Teste no terminal

No Workbench do BlueJ, cria duas criaturas com os mesmos valores. Compara-as através de ==, altera apenas uma delas e observa os estados.

Parte 2 Referências e método clone

Atribuir não significa copiar

```
CriaturaMagica original = new CriaturaMagica("Faísca", "Dragão", 4, 80);
CriaturaMagica supostaCopia = original;

supostaCopia.setEnergia(10);
```

A variável `supostaCopia` recebe a mesma referência. Existe apenas um objeto e duas variáveis apontam para ele. Por isso, a alteração também aparece quando consultamos `original`.

Clone explícito

Neste projeto não será usado `super.clone` nem a interface `Cloneable`. O método cria explicitamente uma nova instância e copia os valores para esse novo objeto.

```
public CriaturaMagica clone()
{
    return new CriaturaMagica(/* valores do objeto atual */);
}
```

Exercício 4 Construir o clone

10. Implementa clone na classe `CriaturaMagica`.
11. O método deve devolver uma nova `CriaturaMagica` com os mesmos valores do objeto atual.
12. Não devolvas `this` e não devolvas uma variável que apenas referencia o objeto atual.

Exercício 5 Provar a independência

13. Cria original, referencia e copia.
14. Faz `referencia = original` e `copia = original.clone()`.
15. Apresenta os resultados de `original == referencia` e `original == copia`.
16. Altera a energia de referencia e depois a energia de copia.
17. Explica por que motivo apenas algumas alterações afetam original.

Previsão antes de executar

```
CriaturaMagica x = new CriaturaMagica("Eco", "Fénix", 3, 70);
CriaturaMagica y = x;
CriaturaMagica z = x.clone();
y.setNivel(8);
z.setEnergia(5);
System.out.println(x.getNivel());
System.out.println(x.getEnergia());
System.out.println(x == y);
System.out.println(x == z);
```

Regista primeiro a tua previsão e só depois confirma no BlueJ.

Parte 3 Um objeto como atributo de outra classe

Composição

Um atributo não tem de ser um número, texto ou valor lógico. A classe Treinador pode guardar uma CriaturaMagica completa.

```
public class Treinador
{
    private String nome;
    private int medalhas;
    private CriaturaMagica companheira;
}
```

O construtor recebe um objeto já criado. A decisão importante é escolher se guarda a referência recebida ou uma cópia independente.

Exercício 6 O treinador

18. Cria a classe Treinador com nome, medalhas e companheira.
19. Cria um construtor completo e um construtor que receba apenas o nome.
20. No construtor completo, guarda um clone da criatura recebida.
21. Cria getNome, setNome, getMedalhas e setMedalhas.
22. Cria getCompanheira e setCompanheira usando clones.
23. Cria ganharMedalha e toString.

Cópia defensiva

Se o getter devolver diretamente companheira, outro código pode alterar o objeto interno do treinador. Devolver um clone protege o estado interno.

```
public CriaturaMagica getCompanheira()
{
    return /* cópia independente */;
}
```

Exercício 7 Testar a proteção

24. Cria uma criatura e utiliza-a para construir um treinador.
25. Altera a criatura original depois de construir o treinador.
26. Confirma se a companheira interna do treinador foi alterada.
27. Obtém a companheira através do getter e altera o objeto devolvido.
28. Confirma novamente se o estado interno do treinador se manteve protegido.

Parte 4 Objetos com ArrayList e cópia profunda

A guilda

A GuildaMágica representa um grupo de criaturas. A lista também é um objeto. Copiar apenas a lista não chega: os elementos guardados podem continuar partilhados.

```
private ArrayList<CriaturaMágica> criaturas;
```

Exercício 8 Construir a guilda

- 29. Cria GuildaMágica com nome e ArrayList de criaturas.
- 30. Inicializa sempre a lista no construtor.
- 31. Cria adicionarCriatura, removerCriatura, quantidade e toString.
- 32. Ao adicionar, guarda um clone da criatura recebida.

Exercício 9 Procurar e verificar

- 33. Cria existeCriatura(String nome), devolvendo true quando encontrar o nome.
- 34. Cria procurarCriatura(String nome), devolvendo um clone da criatura encontrada ou null.
- 35. Cria criaturaMaisForte, devolvendo um clone da criatura com maior nível.
- 36. Compara nomes sem distinguir maiúsculas de minúsculas.

Cópia superficial e cópia profunda

Uma cópia superficial cria outra GuildaMágica, mas pode manter a mesma lista ou as mesmas criaturas. Uma cópia profunda cria a nova guilda, uma nova lista e uma nova criatura para cada elemento.

Exercício 10 Clone profundo da guilda

- 37. Cria o método clone na GuildaMágica.
- 38. Cria primeiro uma guilda nova.
- 39. Percorre a lista original e adiciona uma cópia de cada criatura.
- 40. Altera depois a guilda clonada e demonstra que a original não muda.

Parte 5 Missões entre objetos

Uma classe que recebe objetos

A MissaoMagica recebe um Treinador e uma CriaturaMagica adversária. Assim, uma missão relaciona objetos que já existem.

```
public class MissaoMagica
{
    private String titulo;
    private Treinador treinador;
    private CriaturaMagica adversario;
    private int nivelMinimo;
}
```

Exercício 11 A missão

41. Cria os quatro atributos indicados.
42. No construtor, recebe os objetos e guarda cópias independentes.
43. Cria podelniciar. A missão pode começar quando existe uma companheira, esta tem energia suficiente e o seu nível atinge o mínimo.
44. Cria preverResultado. Compara o nível da companheira com o nível do adversário.
45. Cria toString para descrever a missão.

Exercício 12 Clone da missão

46. Cria um método clone em Treinador.
47. Cria depois clone em MissaoMagica.
48. Garante que a nova missão tem um novo treinador e novas criaturas.
49. Demonstra no programa de teste que nenhuma alteração à cópia afeta a missão original.

Desafio final Academia de Criaturas Mágicas

Cria a classe AppAcademia com o método main. O programa deve construir e testar todo o modelo sem pedir dados ao utilizador.

Requisitos

50. Criar pelo menos quatro criaturas diferentes.
51. Criar dois treinadores e atribuir-lhes companheiras.
52. Criar uma guilda e adicionar pelo menos três criaturas.
53. Pesquisar uma criatura existente e outra inexistente.
54. Criar uma missão com treinador, adversário e nível mínimo.
55. Apresentar o resultado previsto da missão.
56. Clonar uma criatura, um treinador, uma guilda e uma missão.
57. Alterar os clones e provar, através do terminal, que os originais não mudaram.
58. Usar toString para apresentar objetos de forma legível.

Questões de reflexão

59. Qual é a diferença entre `b = a` e `b = a.clone()`?
60. Por que motivo `return this` não produz um clone?
61. Por que razão o getter de um atributo composto pode devolver um clone?
62. O que faltaria copiar se a guilda criasse apenas uma nova ArrayList?
63. Em que situação uma cópia profunda é preferível a uma cópia superficial?

Lista de verificação

- Cada classe pública está no respetivo ficheiro.
- Os atributos estão privados.
- Os construtores deixam os objetos num estado válido.
- Os métodos clone devolvem novos objetos.
- As classes compostas clonam também os seus objetos internos.
- A aplicação de teste demonstra e explica os resultados.